01 / PROPOSTA TÉCNICA

Fynno

Versão 0.0.0 alpha

Proposta funcional e modelagem de dados

Documento de referência para implementação manual em curto prazo. Elaborado em 31/10/2026, a partir dos requisitos apresentados por Fernando.

Objetivo: registrar receitas e despesas, visualizar o compromisso financeiro de cada mês e identificar rapidamente saldo negativo, contas em aberto e compromissos futuros.

Estrutura proposta

Camada	Responsabilidade
Java / Spring Boot	API REST, autenticação, regras, geração de parcelas e recorrências, consolidação financeira.
JavaScript / ReactJS	Login, dashboard anual, detalhe mensal, formulários e cadastros.
MySQL	Persistência relacional, chaves estrangeiras, valores monetários exatos e integridade dos registros.

Escopo da primeira entrega

• Um usuário autenticado é o owner dos seus dados. Pessoas são cadastros auxiliares, sem login próprio.

• Dashboard por ano; detalhe por mês; receitas e despesas à vista, parceladas ou recorrentes; baixa integral de pagamento/recebimento.

• Cadastro de pessoas, formas de pagamento e cartões; compras vinculadas às faturas; filtros por pessoa e situação.

• Saldo inicial e indicadores simples de integridade: saldo projetado, valor em aberto e contas vencidas.

Limites para manter a alpha pequena

Sem integração bancária, importação de faturas, juros de cartão, pagamento parcial, conciliação, múltiplas moedas, rateio de uma mesma compra entre pessoas ou compartilhamento de acesso. Todas as quantias são em BRL. Essas funções podem ser adicionadas depois.

Entrega desta etapa: proposta e MER. Este documento não representa uma API ou frontend já implementados. As decisões abaixo são sugestões de escopo, não requisitos previamente confirmados.

02 / PROPOSTA TÉCNICA

Ajustes nas regras de negócio

Ponto original	Decisão sugerida para a alpha
Crédito, débito e saldo	Usar RECEITA e DESPESA na API e nos formulários. “Crédito” no dashboard significa entrada, não compra no cartão.
Parcelado: valor e número de período	Informar valor TOTAL da compra, quantidade de parcelas mensais e primeiro vencimento ou primeira fatura. Exibir também o valor calculado por parcela.
Recorrente semelhante a uma conta à vista	Recorrente exige primeiro vencimento, frequência mensal e fim opcional. O valor informado é o valor de cada ocorrência.
Vencimento só com o dia	Persistir a data completa. O dia é apenas uma apresentação dentro do mês. Dias 29, 30 e 31 são ajustados ao último dia válido.
“Pago?” para toda conta	Usar “Pago?” nas despesas e “Recebido?” nas receitas. A marcação registra uma data de liquidação integral.
Cartão e fatura no fluxo	Compra é o detalhamento; fatura é a saída no fluxo consolidado. Nunca somar os dois como despesas independentes.
Pessoa aparece com seu saldo	Saldo é um resumo virtual das contas dela. Não criar outro lançamento no banco para esse resumo.
Saldo devedor da pessoa	Adotar saldo = receitas - despesas: negativo reduz o saldo do owner; positivo aumenta. Não inverter o sinal automaticamente.

O ponto que exige uma escolha conceitual

A proposta trata cada pessoa como uma **divisão do orçamento administrado pelo owner**. Suas receitas e despesas integram o orçamento consolidado. Portanto, déficit de uma pessoa não é automaticamente um valor que ela deve reembolsar ao owner.

> Se “saldo devedor” significar **dinheiro emprestado e a receber da pessoa**, será necessário um controle de adiantamentos e acertos. Registrar um reembolso automático a partir de qualquer déficit criaria receitas sem um recebimento real. Para a alpha, recomendo a interpretação de subcontas de orçamento e manter acertos fora do escopo.

Essa definição preserva a regra de pessoa opcional: pessoa_id vazio pertence ao owner. Pessoa vinculada identifica somente a subdivisão responsável; user_id continua sendo o proprietário dos dados.

03 / PROPOSTA TÉCNICA

Saldos, pessoas e cartão sem duplicidade

Duas leituras do mesmo orçamento

Visão por pessoa: apresenta receitas, despesas diretas e parcelas de cartão atribuídas àquela pessoa, no mês em que a fatura vence. O saldo mostrado é calculado a partir desses itens. O grupo “Minhas contas” reúne pessoa_id vazio.

Visão consolidada do owner: soma todas as receitas e despesas diretas, de todas as pessoas, e usa uma linha para cada fatura completa. As compras do cartão ficam disponíveis ao expandir a fatura. Os resumos das pessoas são informativos nesta visão e não são somados novamente.

Regra de agregação: ou somar os grupos por pessoa com suas parcelas de cartão, ou somar despesas diretas e faturas completas. Os dois caminhos devem dar o mesmo saldo. Misturar os dois caminhos gera duplicidade.

Exemplo de novembro

Origem	Receitas	Despesas diretas	Cartão	Saldo
Minhas contas	5.000,00	2.000,00	700,00	2.300,00
Pessoa Ana	500,00	200,00	300,00	0,00
Consolidado	5.500,00	2.200,00	1.000,00	2.300,00

Na lista consolidada: receitas de R$ 5.500, despesas diretas de R$ 2.200 e uma fatura de R$ 1.000. Resultado: R$ 2.300. O saldo de Ana é zero, mas suas despesas e receitas continuam visíveis no detalhamento. Se ela tivesse R$ 100 de déficit, reduziria o consolidado em R$ 100.

Mês, acumulado e representatividade

• **Saldo mensal projetado:** receitas previstas do mês - despesas previstas do mês, incluindo itens quitados. Filtrar somente os abertos produziria outro indicador, chamado “em aberto”.

• **Saldo acumulado projetado:** saldo inicial + soma dos saldos mensais desde a data-base até o mês selecionado, inclusive anos anteriores.

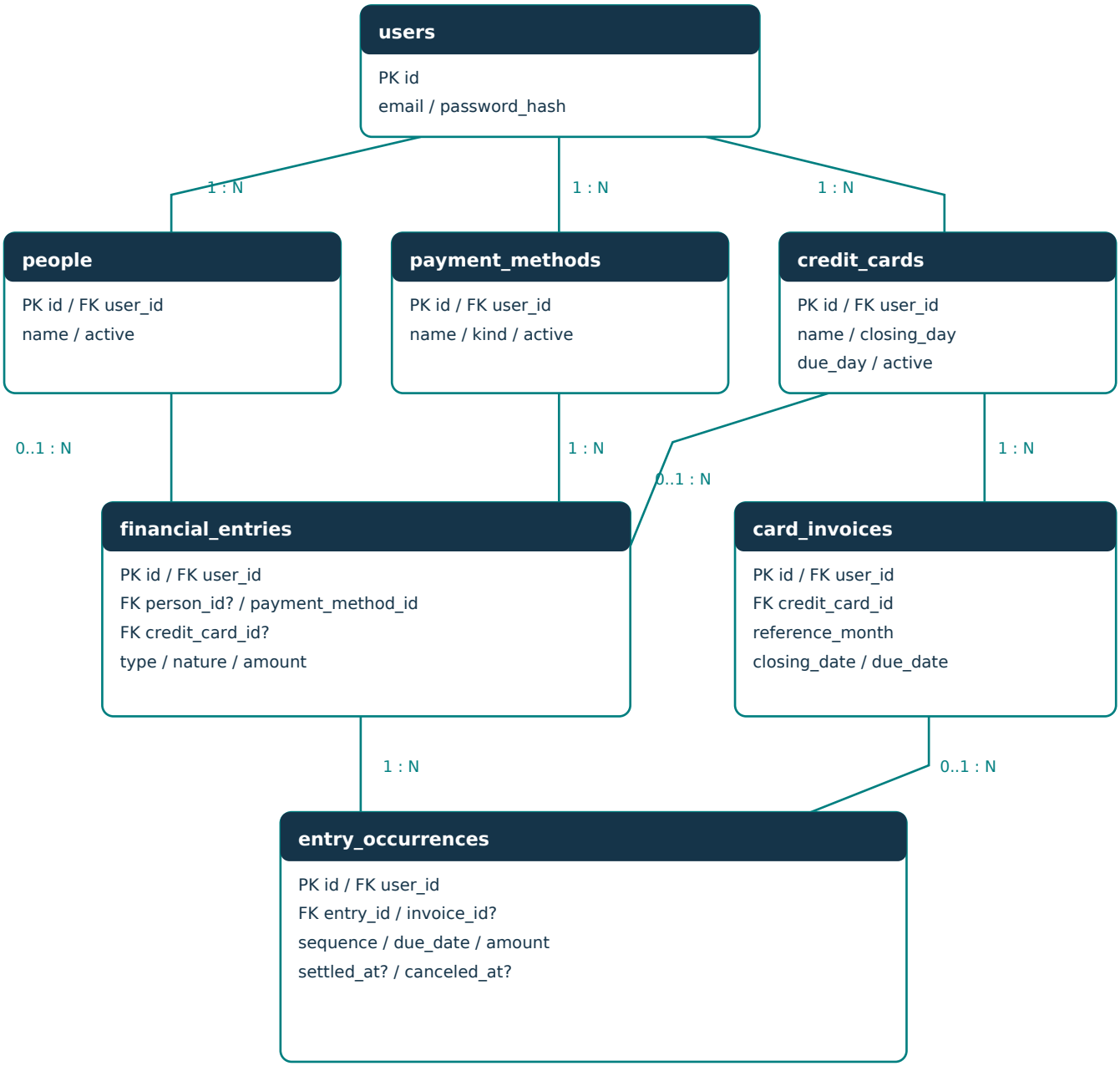
• **Realizado:** somar somente baixas e faturas pagas, pela data efetiva da liquidação. Não misturar esse resultado com o projetado por vencimento.

• **Representatividade:** despesa ÷ total de despesas do mês; receita ÷ total de receitas do mês. Aplicar o filtro de pessoa e a mesma base de agregação. Denominador zero resulta em 0%.

Percentuais de compras dentro de uma fatura usam o total da própria fatura e são identificados como “% da fatura”. Não entram na soma percentual das linhas principais.

04 / PROPOSTA TÉCNICA

MER: sete tabelas, um único owner



Legenda: PK = chave primária; FK = chave estrangeira; ? = campo opcional. Em uma relação “0..1 : N”, cada registro do lado N pode estar ligado a zero ou um registro do lado oposto; o registro oposto pode ter vários filhos. Todo lado N admite zero filhos.

Vínculos de propriedade: users também tem relação 1:N direta com financial_entries, card_invoices e entry_occurrences, por user_id. Essas três linhas foram omitidas apenas do desenho para facilitar a leitura; as FKs são obrigatórias.

Não há tabela de dashboard, saldo mensal, saldo de pessoa ou total de fatura. São resultados calculados. Também não há tabela de “conta do owner”: users representa o único proprietário na alpha.

05 / PROPOSTA TÉCNICA

Dicionário: acesso e cadastros

Convenções: id BIGINT PK; user_id BIGINT NOT NULL FK users.id; datas de negócio DATE; eventos DATETIME em UTC; booleanos BOOLEAN; textos em utf8mb4. Todas as tabelas possuem created_at e updated_at DATETIME. Campos são obrigatórios, exceto os marcados com ?.

users - proprietário e autenticação

Campo / tipo	Regra
id / BIGINT	Identificador primário.
name / VARCHAR(120)	Nome do usuário.
email / VARCHAR(190)	Único; normalizar antes de gravar.
password_hash / VARCHAR(255)	Hash da senha; nunca armazenar senha legível.
opening_balance / DECIMAL(15,2)	Saldo inicial, admite positivo ou negativo; padrão 0.
balance_start_date / DATE	Data-base do saldo inicial. Primeiro dia de um mês; aberturas do mês passam a compor os saldos a partir dessa data.

people - pessoas sem credenciais

Campo / tipo	Regra
id; user_id / BIGINT	Pessoa pertence a um único owner.
name / VARCHAR(120)	Nome de exibição; não exige unicidade.
active / BOOLEAN	Padrão true. Inativa não recebe novas contas; histórico continua visível.

payment_methods - formas de pagamento

Campo / tipo	Regra
id; user_id / BIGINT	Cadastro do owner; não representa conta bancária.
name / VARCHAR(80)	Ex.: Pix, dinheiro, boleto, cartão de crédito.
kind / VARCHAR(20)	DIRECT ou CREDIT_CARD. DIRECT cobre pagamentos e recebimentos fora de cartão.
active / BOOLEAN	Padrão true; inativação preserva o histórico.

credit_cards - cartões do owner

Campo / tipo	Regra
id; user_id / BIGINT	Todo cartão pertence ao owner.
name / VARCHAR(80)	Apelido; não guardar número completo, CVV ou senha.
closing_day; due_day / TINYINT	Valores de 1 a 31. Aplicar ajuste ao último dia do mês.
active / BOOLEAN	Inativar impede novas compras e preserva faturas existentes.

06 / PROPOSTA TÉCNICA

Dicionário: origem dos lançamentos

financial_entries - conta ou série

Representa a intenção original: uma conta à vista, uma compra parcelada ou uma regra recorrente. Não é somada diretamente no dashboard. Os valores financeiros vêm das ocorrências.

Campo / tipo	Finalidade e validação
id; user_id / BIGINT	PK e owner obrigatório.
person_id? / BIGINT	FK people; nulo = owner. Toda a série é de uma única pessoa.
payment_method_id / BIGINT	FK payment_methods; obrigatório.
credit_card_id? / BIGINT	FK credit_cards; obrigatório apenas para forma CREDIT_CARD.
description / VARCHAR(255)	Descrição da conta ou compra.
nature / VARCHAR(10)	INCOME ou EXPENSE. Valores sempre positivos; natureza define o sinal.
type / VARCHAR(15)	ONE_TIME, INSTALLMENT ou RECURRING.
amount / DECIMAL(15,2)	À vista: valor da conta. Parcelado: total da compra. Recorrente: valor de cada ocorrência.
purchase_date / DATE	Data do lançamento/compra; não define sozinha o mês do dashboard.
first_due_date / DATE	Primeiro vencimento. Em cartão, vencimento da primeira fatura escolhida.
installment_count? / SMALLINT	Somente parcelado, mínimo 2. Na alpha as parcelas são mensais.
recurrence_end_date? / DATE	Fim inclusivo da recorrência; nulo = sem término. Considerar a data efetiva ajustada da ocorrência.
canceled_at? / DATETIME	Interrompe a série. Não apaga contas quitadas nem registros anteriores.

Validações condicionais

• amount > 0; INSTALLMENT exige installment_count e não aceita recurrence_end_date. Nos demais tipos, installment_count deve ser nulo.

• RECURRING é mensal. Para limitar o escopo inicial, recorrência e cartão não se combinam na alpha; assinaturas no cartão podem ser lançadas à vista em cada fatura.

• CREDIT_CARD exige nature = EXPENSE. Receitas e recebimentos ficam em DIRECT; estornos e créditos em fatura ficam para uma evolução.

• Pessoa e forma de pagamento ficam fixas na série. Depois de existir baixa ou fatura fechada, não trocar sua atribuição: corrigir somente itens futuros ou criar outra série.

07 / PROPOSTA TÉCNICA

Dicionário: ocorrências e faturas

entry_occurrences - cada obrigação individual

Campo / tipo	Finalidade e validação
id; user_id / BIGINT	PK e owner obrigatório.
entry_id / BIGINT	FK financial_entries. Uma conta à vista também tem uma ocorrência.
invoice_id? / BIGINT	FK card_invoices. Preenchido apenas em compras no cartão.
sequence / INT	1 na conta à vista; 1..N no parcelado; sequência mensal na recorrência.
due_date / DATE	Vencimento integral; em cartão deve ser igual ao da fatura.
amount / DECIMAL(15,2)	Valor efetivo da parcela/ocorrência; sempre maior que zero.
description_override? / VARCHAR(255)	Descrição própria; nulo herda a descrição da série.
settled_at? / DATETIME	Baixa integral para DIRECT. Em cartão deve ser nulo: a baixa vem da fatura.
canceled_at? / DATETIME	Exclui dos totais. Não é permitido cancelar uma ocorrência já liquidada.

card_invoices - agrupamento das compras

Campo / tipo	Finalidade e validação
id; user_id / BIGINT	PK e owner obrigatório.
credit_card_id / BIGINT	FK credit_cards; apenas um cartão por fatura.
reference_month / DATE	Primeiro dia do mês de vencimento. Único por cartão.
closing_date; due_date / DATE	Datas efetivas do ciclo. closing_date deve ser anterior a due_date. São preservadas mesmo que o cadastro do cartão mude.
closed_at? / DATETIME	Marca o fechamento e bloqueia alteração das compras desta fatura.
paid_at? / DATETIME	Baixa integral; exige fatura fechada. Todas as compras herdam o estado de pagamento da fatura.

Total da fatura = soma das ocorrências não canceladas vinculadas. Não gravar outra despesa para a fatura. Pagar altera paid_at; não cria nova saída adicional. Faturas vazias não aparecem no fluxo nem podem ser pagas.

Situação é calculada: cancelado; liquidado; vencido (em aberto e vencimento anterior a hoje); em aberto. Para fatura, apresentar também aberta/fechada. Não persistir um booleano pago em duplicidade.

08 / PROPOSTA TÉCNICA

Geração, edição e pagamento

À vista e parcelado

Salvar a série e suas ocorrências na mesma transação. À vista gera uma ocorrência. Parcelado gera todas as parcelas de uma vez. O formulário informa explicitamente que o valor digitado é o total, evitando multiplicá-lo indevidamente.

Arredondamento: calcular em centavos. Em R$ 100 ÷ 3, gerar R$ 33,33, R$ 33,33 e R$ 33,34. Rejeitar quantidade de parcelas maior que o valor em centavos. A soma das ocorrências deve ser exatamente o total original.

Recorrências mensais sem registros infinitos

Gerar até dezembro do maior entre o ano atual e o ano consultado. Ao acessar outro ano, completar somente os meses ausentes desde o início da série. Executar o preenchimento pelo serviço de consulta, dentro de transação, com unicidade por série/sequência; não exigir scheduler na alpha. Consultar um ano distante pode ter limite configurável na API.

Derivar cada vencimento da data original, nunca do vencimento anterior já ajustado. Ex.: 31/01 → 28/02 → 31/03. A geração respeita o fim inclusivo e o cancelamento. O mês consultado deve incluir todas as séries aplicáveis antes do cálculo dos totais.

Cartão e competência da fatura

O usuário escolhe a **primeira fatura** ao lançar a compra. As demais parcelas vão para as faturas dos meses seguintes. A data da compra ajuda a sugerir a primeira fatura, mas o usuário pode corrigir a sugestão; essa escolha prevalece sobre regras particulares do banco.

reference_month identifica o mês do vencimento. Gerar due_date com o dia configurado; closing_date é a ocorrência mais recente do dia de fechamento estritamente anterior a due_date. Ajustar dias inválidos. Essas datas são snapshots; alterações no cartão afetam apenas novas faturas. Não aceitar novas compras em faturas fechadas ou pagas.

Edição e cancelamento

• Ao clicar na descrição, abrir a ocorrência. Para à vista, editar o item. Para série, oferecer “somente esta” e “esta e futuras em aberto”.

• Edição isolada altera description_override, amount ou due_date. Em cartão, mudar vencimento significa mover para outra fatura aberta, nunca deixar data divergente.

• Parcelado já iniciado: manter número e total contratado. Ajuste de valor deve redistribuir o restante entre parcelas editáveis, sem alterar quitadas ou faturas fechadas; rejeitar quando isso for impossível.

• Recorrente: alterar a série para as futuras ainda não geradas e atualizar as ocorrências futuras em aberto já existentes. Cancelar a série também cancela as futuras em aberto.

• Baixa e reversão são integrais. Reverter fatura limpa paid_at e mantém o fechamento. Reabrir exige ausência de pagamento. Histórico anterior à edição é mantido para itens quitados.

09 / PROPOSTA TÉCNICA

Telas e contratos da API

Tela	Comportamento esperado
Login	E-mail e senha. Sessão autenticada; nunca aceitar owner informado pelo frontend.
Dashboard anual	12 cartões mensais com receitas, despesas e saldo projetado. Indicar mês negativo e vencidos. Selecionar ano e pessoa.
Detalhe mensal	Vencimento, representatividade, descrição, valor e pago/recebido. Mostrar saldo mensal, acumulado e totais em aberto.
Modo consolidado	Faturas aparecem por total, com compras expansíveis; saldos de pessoas ficam em resumo informativo.
Modo por pessoa	Contas diretas e parcelas de cartão daquela pessoa. Exibir a fatura de origem; pagar cartão somente pela fatura completa.
Novo lançamento / edição	Natureza, pessoa opcional, tipo, descrição, valor, pagamento e vencimento/primeira fatura. Campos específicos para parcelas e recorrência.
Cadastros e configuração	Pessoas, formas de pagamento, cartões, saldo inicial e data-base.

Rotas sugeridas, sem multiplicar serviços

Recurso	Operações essenciais
/api/auth	POST /login; POST /logout; GET /me. Provisionar o primeiro usuário; cadastro público fica opcional.
/api/people; /payment-methods; /credit-cards	GET, POST, PATCH /{id}. Inativar em vez de remover registros usados.
/api/entries	POST cria série e ocorrências; GET /{id}; PATCH /{id} para a regra; POST /{id}/cancel.
/api/occurrences	GET por mês/pessoa; PATCH /{id} com escopo THIS ou THIS_AND_FUTURE; PUT /{id}/settlement e DELETE /{id}/settlement.
/api/invoices	GET por mês/cartão; GET /{id}; PUT e DELETE /{id}/closure; PUT e DELETE /{id}/payment.
/api/dashboard	GET ?year=2026; GET /months/2026-11. Filtros personId e scope=ALL|OWNER|PERSON.
/api/settings	GET e PATCH para saldo inicial e data-base.

Rotas abreviadas de cadastros compartilham o prefixo /api. Valores monetários podem trafegar como strings decimais, evitando arredondamento no JavaScript. A API devolve totais prontos; o React usa esses totais para exibição.

10 / PROPOSTA TÉCNICA

Integridade e implantação enxuta

Restrições e índices do modelo físico

• UNIQUE users(email); UNIQUE card_invoices(credit_card_id, reference_month); UNIQUE entry_occurrences(entry_id, sequence). reference_month sempre no primeiro dia do mês.

• Índices: financial_entries(user_id, person_id); entry_occurrences(user_id, due_date); entry_occurrences(invoice_id); card_invoices(user_id, due_date). Acrescentar apenas conforme consultas reais.

• Todo vínculo deve preservar user_id. Usar UNIQUE(user_id, id) nos pais e FK composta (user_id, referência) nas tabelas filhas. As FKs opcionais aceitam referência nula.

• Além das FKs, validar na transação: cartão da série = cartão da fatura; forma DIRECT implica cartão e fatura nulos; CREDIT_CARD exige ambos; vencimento da ocorrência = vencimento da fatura.

• Usar DECIMAL no banco e BigDecimal no Java. Não calcular dinheiro com float/double. Proteger geração e alterações concorrentes com transação, unicidade e bloqueio da série/fatura quando necessário.

• Não usar exclusão em cascata em dados financeiros. Remover cadastro somente se nunca usado; caso contrário, inativar. Cancelamentos preservam linhas. O saldo inicial representa somente a posição anterior à data-base: ocorrências anteriores ficam fora do acumulado.

Organização técnica suficiente para começar

Um backend Spring Boot como monólito, dividido em auth, cadastros, lançamentos, faturas e dashboard. Controllers chamam serviços transacionais; repositories cuidam da persistência. React com rotas e formulários simples. Migrations versionadas definem o banco; evitar atualização automática do schema em produção.

Sessão com cookie HttpOnly é uma opção simples quando frontend e API usam o mesmo domínio por proxy. Configurar HTTPS, proteção CSRF para mutações e acesso autenticado. Buscar sempre os dados pelo owner da sessão. Nunca confiar somente em um id recebido na URL.

Ordem sugerida de implementação

Etapa	Resultado verificável
1. Base	Migrations, usuário inicial, autenticação e isolamento por owner.
2. Núcleo	Cadastros, lançamento à vista, baixa e listagem mensal.
3. Projeções	Parcelamento, recorrência, edição de séries e cancelamento.
4. Cartão	Faturas, vínculo das compras, fechamento e pagamento.
5. Visões	Dashboard anual, agrupamento por pessoa, acumulado e indicadores.

11 / PROPOSTA TÉCNICA

Critérios de aceite e próximos passos

Cenários que validam a alpha

Cenário	Resultado esperado
Conta sem pessoa	Aparece em “Minhas contas”; owner vem da sessão.
Receita 5.000; despesa 2.000	Saldo projetado 3.000, tanto antes quanto depois das baixas.
Compra 100 em 3 parcelas	33,33 + 33,33 + 33,34, em três meses; soma exata 100.
Parcela em dezembro	Demais parcelas seguem em janeiro e fevereiro do ano seguinte.
Recorrência no dia 31	Fevereiro usa o último dia; março volta ao dia 31. Consultas repetidas não duplicam ocorrências.
Compra de 300 em cartão	Total da fatura sobe 300; despesa consolidada sobe somente 300.
Fatura com compras de duas pessoas	Soma das partes bate com o total da fatura; consolidado e soma dos grupos têm o mesmo saldo.
Pessoa com receita 100 e despesa 250	Saldo -150; reduz o consolidado em 150. Não cria crédito de reembolso automático.
Pagamento antecipado	Projeção continua no vencimento; realizado aparece no mês da data efetiva.
Baixa e reversão	Repetir a mesma baixa não duplica efeito; reverter restaura o estado em aberto.
Acesso a id de outro usuário	Leitura e alteração bloqueadas, inclusive para vínculos indiretos.
Fatura fechada ou paga	Compra não pode ser alterada; reversão/reabertura seguem o fluxo definido.

Pitacos simples que ajudam desde o início

• Mostrar “valor total” e a prévia das parcelas antes de salvar. É uma proteção barata contra lançamento errado.

• Oferecer “duplicar conta” para agilizar cadastro, sem criar automações adicionais.

• Usar filtros “em aberto”, “vencidas” e “quitadas”, mantendo o total projetado identificado na tela.

• Adicionar categorias depois do núcleo funcionar; orçamento por categoria e metas podem esperar.

Recomendação: começar por à vista + dashboard mensal, mas criar desde o início a separação entre série e ocorrência. Ela sustenta parcelamento e recorrência sem exigir refazer o banco. Antes do código, confirmar somente o significado de saldo de pessoa: orçamento administrado ou dívida de reembolso.

Modelagem elaborada especificamente para os requisitos apresentados. Nomes de tabelas, campos, rotas e limitações são uma proposta de implementação para a 0.0.0 alpha.